

Video RAG Retrieval Project

<https://github.com/aiyshwary/Video-RAG-Retrieval>

OVERVIEW

Python **FAISS** **OpenCLIP** **ViT-GPT2** **Sentence Transformers** **RAG**
A semantic video search system that extracts frames, generates scene captions using ViT-GPT2, and en

IMPACT

Built a Visual RAG system that makes unstructured video content semantically searchable: a text query r

TECHNICAL WALKTHROUGH

One-line summary

I built a Visual Retrieval-Augmented Generation (RAG) system that retrieves relevant video scenes bas

Problem statement

Searching videos using text is hard because video content is unstructured. Traditional keyword search

High-level architecture

i) Scene understanding using vision models

ii) Vector storage for text and image embeddings

iii) Similarity-based retrieval using VDMS

iv) Video → Scene Description + Frames → Embeddings → Vector DB → Retrieval → Video Clips

Input & preprocessing

The input to the system is one or more videos. Each video is processed to extract: Scene-level textual

Vision understanding

Vision-language models are used to understand the video content. These models analyze video scene

Embedding strategy

To support efficient retrieval, two types of embeddings are stored in separate vector stores:

i) Text embeddings: Model: SentenceTransformer – all-MiniLM-L6-V2 / L12-V2. Used for scene descriptions.

ii) Image embeddings: Model: OpenCLIP – ViT-g-14. Used for video frames. This helps capture visual context.

Vector storage & retrieval

Both text and image embeddings are stored in VDMS, which allows fast vector similarity search. Separation of metadata and embeddings improves scalability.

i) Stored metadata includes: Video name, Scene ID, Frame number, Timestamp

Query flow

When a user enters a text prompt: Prompt is converted into a text embedding. Vector similarity search finds relevant video segments.

Final output

Scene description and exact time range to jump to in the video. Text embeddings capture semantic meaning, while image embeddings provide visual context.

Example

i) 'a red car' → image embeddings

ii) 'a tense conversation' → text embeddings

Technical depth

i) Efficient vector search using cosine similarity

ii) Decoupling embeddings for scalability

iii) Model choice based on speed vs accuracy

iv) Frame sampling using FPS to avoid redundancy

Closing line

This approach makes video content searchable, explainable, and scalable, and it can be extended to s

Short version

A Visual RAG system enables semantic search over videos. It uses vision models to generate scene d

KEY DETAILS

1. Extracts frames at a configurable FPS and generates natural-language scene captions using ViT-GP

2. Computes text embeddings with SentenceTransformers (MiniLM) and image embeddings with Open

3. Stores embeddings and metadata in two parallel FAISS indexes supporting independent text or ima

4. Query interface accepts a natural language string and returns top-k matching scenes with frame refe

5. Modular architecture (preprocessing, embeddings, storage, retrieval) designed for extension to prod

6. FAISS indexes use IndexFlatIP with L2-normalized vectors for cosine-similarity search; OpenCLIP o

7. Includes a pytest suite and a self-contained demo that synthesizes a 30-frame test video with Open